

Performance Analysis

[Reattempt](#)[Download Certificate](#)

Detailed analysis of your performance in the AWS Certified Solutions Architect Associate - SAA C03 exam.

Answer Distribution

0

Correct

0

Incorrect

44

Unattempted

Domain Performance

Security	0%
Cost Optimization	0%
Databases	0%
Compute	0%
Storage	0%
Networking	0%
Serverless	0%

**0/44**

Passing score: 33

Time Taken

**130m 0s**

Total time: 130 minutes

Question Analysis

Correct

0

Incorrect

0

Unattempted

44

Reviewed

0

Question-wise Analysis

Expand All

Collapse All

Question 1

Not Attempted

Security



An Amazon EC2 administrator created the following IAM policy associated with an IAM group containing several users:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:TerminateInstances",
      "Resource": "*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "10.100.100.0/24"
        }
      }
    },
    {
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "ec2:Region": "us-east-1"
        }
      }
    }
  ]
}
```

examtopics.io

What is the effect of this policy?

- ☐ Users can terminate an EC2 instance in any AWS Region except us-east-1.
This is incorrect. The policy allows termination in us-east-1 under certain conditions.
- ☐ Users can terminate an EC2 instance with the IP address 10.100.100.1 in the us-east-1 Region.
While 10.100.100.1 falls within the CIDR block, certain IPs are reserved within the VPC, so this isn't universally true. Furthermore the DENY statement takes precedence.
- ☐ Users can terminate an EC2 instance in the us-east-1 Region when the user's source IP is 10.100.100.254.
The `Deny` statement restricts operations to only the us-east-1 region. The `Allow` statement permits termination from the 10.100.100.0/24 CIDR block. However, the `Deny` will ultimately take precedence.
- ☒ Users cannot terminate an EC2 instance in the us-east-1 Region when the user's source IP is 10.100.100.254.
The policy allows users to terminate EC2 instances only when their source IP is within the range 10.100.100.0/24. However, there is a Deny statement that blocks users from terminating any EC2 instance in regions other than us-east-1. Since the second statement has a higher priority than the first statement, users who have a source IP address of 10.100.100.254 will not be able to terminate an EC2 instance in the us-east-1 region.

Overall Explanation:

The policy contains an Allow statement that permits termination if the source IP is within 10.100.100.0/24. It also contains a Deny statement that restricts all EC2 actions to only the us-east-1 Region. Deny statements always take precedence over Allow statements. Thus users cannot terminate an EC2 instance in the us-east-1 Region when the user's source IP is 10.100.100.254. because the request will be denied.

[View Discussions](#)[Ask AI](#)

Question 2

Not Attempted

Cost Optimization



A consumer survey company has gathered data for several years from a specific geographic region. The company stores this data in an Amazon S3 bucket in an AWS Region. The company has started to share this data with a marketing firm in a new geographic region. The company has granted the firm's AWS account access to the S3 bucket. The company wants to minimize *its own* data transfer costs when the marketing firm requests data from the S3 bucket. Which solution will meet these requirements?



Configure the Requester Pays feature on the company's S3 bucket.

With Requester Pays enabled, the marketing firm's AWS account becomes responsible for the data transfer costs when accessing data in the S3 bucket, minimizing costs for the survey company.



Configure S3 Cross-Region Replication (CRR) from the company's S3 bucket to one of the marketing firm's S3 buckets.

CRR replicates data, but the survey company bears the initial transfer and storage costs. This doesn't minimize *their* costs when the marketing firm accesses data.



Configure AWS Resource Access Manager to share the S3 bucket with the marketing firm AWS account.

RAM facilitates resource sharing, but it doesn't influence data transfer costs, so the company would still be responsible for data transfer costs.



Configure the company's S3 bucket to use S3 Intelligent-Tiering. Sync the S3 bucket to one of the marketing firm's S3 buckets.

Intelligent-Tiering optimizes storage costs based on access patterns, but it does not reduce data transfer costs for requests made by the marketing firm from a different region. Also the statement

'Sync the S3 bucket' is incorrect and doesn't correspond to the intended functionality of S3 Intelligent Tiering.

Overall Explanation:

The Requester Pays feature allows the survey company to shift the data transfer costs to the marketing firm when they access the data. This fulfills the requirement of minimizing the survey company's data transfer costs. CRR, RAM, and Intelligent-Tiering do not address this specific cost minimization goal for the company.

[View Discussions](#)[Ask AI](#)

Question 3

Not Attempted

Security



A company hosts its public e-commerce website on AWS. The website uses AWS Global Accelerator to manage internet traffic, which is then forwarded to an Application Load Balancer (ALB) serving as the entry point for an Auto Scaling group. Recently, the company experienced a DDoS attack. They need a solution to mitigate future attacks with minimal implementation effort. Which solution should the company implement?

- ☐ ⓧ Configure an AWS WAF web ACL for the Global Accelerator to block traffic using rate-based rules.

While AWS WAF provides protection, it cannot be directly applied to Global Accelerator, which operates at layer 4 (TCP/UDP). Global Accelerator focuses on directing traffic to healthy endpoints but does not inspect the traffic content for web application attacks.

- ☐ ⓧ Configure an AWS Lambda function to read ALB metrics and block attacks by updating a VPC network ACL.

This solution is complex and involves significant implementation effort. Lambda would need to continuously monitor ALB metrics, identify attack patterns, and then dynamically modify network ACLs, making it a cumbersome approach.

- ☒ ⓧ Configure an AWS WAF web ACL on the ALB to block traffic using rate-based rules.

This is the most efficient solution. AWS WAF can be directly integrated with the ALB, allowing for the creation of rate-based rules to block malicious traffic and protect the web application. WAF is designed for layer 7 protection, making it suitable for mitigating web application attacks.

- ☐ ⓧ Configure an Amazon CloudFront distribution in front of the Global Accelerator.

While CloudFront provides DDoS protection and caching, adding it in front of Global Accelerator adds unnecessary complexity and might not be the most cost-effective solution, considering the primary requirement is DDoS mitigation with minimal implementation effort.

Overall Explanation:

Configuring an AWS WAF web ACL on the ALB to block traffic using rate-based rules is the most effective solution with the least implementation effort. AWS WAF is designed to protect web applications against attacks like DDoS by analyzing HTTP/HTTPS traffic and blocking malicious requests based on defined rules. Global Accelerator itself doesn't natively integrate with AWS WAF for traffic inspection. Therefore, placing WAF in front of ALB which handle web requests is the best approach.

[View Discussions](#)[Ask AI](#)

Question 4

Not Attempted

Databases



A company uses an Amazon DynamoDB table to store data received from devices. The DynamoDB table supports a customer-facing website that displays recent activity on customer devices. The table is configured with provisioned throughput for writes and reads. The company wants to calculate performance metrics for customer device data daily. The solution must minimize the effect on the table's provisioned read and write capacity. Which solution will meet these requirements?

- ☐ Use an Amazon Athena SQL query with the Amazon Athena DynamoDB connector to calculate performance metrics on a recurring schedule.

Athena queries DynamoDB directly, which can consume the table's provisioned read capacity. This does not minimize the impact on the DynamoDB table's performance.

- ☒ Use an AWS Glue job with the AWS Glue DynamoDB export connector to calculate performance metrics on a recurring schedule.

The AWS Glue DynamoDB export connector exports a snapshot of the DynamoDB table to Amazon S3 for processing. This minimizes the impact on the table's provisioned read and write capacity because the performance metrics are calculated on the exported data, not directly on the DynamoDB table.

- ☐ Use an Amazon Redshift COPY command to calculate performance metrics on a recurring schedule.

Using Redshift COPY command to directly load data from DynamoDB is not a direct integration and would require an intermediate storage or processing layer. Redshift is also more suitable for data warehousing and complex analytics, not simple performance metrics calculation.

- ⊗ Use an Amazon EMR job with an Apache Hive external table to calculate performance metrics on a recurring schedule.

Using EMR with Hive requires setting up and managing an EMR cluster, which is more complex than necessary for this use case. Also, accessing DynamoDB directly from Hive can impact the provisioned read capacity.

Overall Explanation:

The AWS Glue DynamoDB export connector offers an efficient method to extract, transform, and load (ETL) data from DynamoDB to Amazon S3, without significantly affecting the table's provisioned throughput. Once the data is in S3, it can be further processed or queried using AWS Glue or other analytics services without impacting DynamoDB's immediate R/W provisioning. This approach minimizes direct read/write operations on the DynamoDB table because the exported data can be processed separately.

 View Discussions

 Ask AI

Question 5

Not Attempted

Compute



A solutions architect is designing a cloud architecture for a new stateless application deployed on AWS. They have created an Amazon Machine Image (AMI) and launch template for the application. The application needs to process jobs in parallel, scaling EC2 instances up and down based on the workload. The application must be loosely coupled, and job items need durable storage. Which solution best meets these requirements?

- ⊗ Create an Amazon Simple Notification Service (Amazon SNS) topic to send the jobs. Create an Auto Scaling group using the launch template with a scaling policy based on CPU utilization.

SNS is used for pub/sub messaging, not durable storage of jobs. Scaling based solely on CPU utilization might not accurately reflect the number of jobs waiting to be processed.

- ⊗ Create an Amazon Simple Queue Service (Amazon SQS) queue to hold the jobs. Create an Auto Scaling group using the launch template with a scaling policy based on

network usage.

SQS is appropriate for durable job storage. However, scaling based on network usage may not directly correlate with the number of jobs needing processing and could lead to inefficient scaling.

- ✓ Create an Amazon Simple Queue Service (Amazon SQS) queue to hold the jobs. Create an Auto Scaling group using the launch template with a scaling policy based on the number of items in the SQS queue.

SQS provides durable storage for the jobs, and scaling the Auto Scaling group based on the queue depth ensures that the number of EC2 instances directly corresponds to the workload.

- ✗ Create an Amazon Simple Notification Service (Amazon SNS) topic to send the jobs. Create an Auto Scaling group using the launch template with a scaling policy based on the number of messages published to the SNS topic.

SNS is not designed for durable storage of job items. Scaling based on the number of messages published to the SNS topic is also not an effective way to scale based on pending job volume.

Overall Explanation:

The best solution is to use Amazon SQS for durable storage of the job items and create an Auto Scaling group that scales based on the number of messages in the SQS queue. This ensures loose coupling, durable storage, and scaling appropriate to the workload. SNS is not the right choice because it is for pub/sub and not durable job storage. Network and CPU utilization are not as effective at dictating scaling as knowing the queue depth, which represents the jobs that need to be processed.

 View Discussions

 Ask AI

Question 6

Not Attempted

Databases



A global e-commerce company currently uses a monolithic architecture. To address increasing product data volume, they need a solution that provides scalability and a modular service architecture while preserving structured database schemas. The solution must also efficiently store both product data and product images with the LEAST operational overhead. Which solution BEST meets these requirements?

- ✗ Use an Amazon EC2 instance in an Auto Scaling group to deploy a containerized application. Use an Application Load Balancer to distribute web traffic. Use an Amazon RDS DB instance to store product data and product images.

This option requires managing EC2 instances and Auto Scaling groups, increasing operational overhead. Storing images directly in RDS is generally not recommended due to performance and storage limitations.

- ⊗ Use AWS Lambda functions to manage the existing monolithic application. Use Amazon DynamoDB to store product data and product images. Use Amazon Simple Notification Service (Amazon SNS) for event-driven communication between the Lambda functions.

While Lambda provides scalability, it's not ideal for managing a large, complex application directly. DynamoDB is a NoSQL database, which doesn't directly support maintaining existing structured database schemas. This would likely require significant refactoring of the data model and application logic.

- ⊗ Use Amazon Elastic Kubernetes Service (Amazon EKS) with an Amazon EC2 deployment to deploy a containerized application. Use an Amazon Aurora cluster to store the product data. Use AWS Step Functions to manage workflows. Store the product images in Amazon S3 Glacier Deep Archive.

EKS and EC2 deployments, while powerful, involve significant operational overhead in managing the Kubernetes cluster and EC2 instances. S3 Glacier Deep Archive is designed for long-term archival, which is not suitable for frequently accessed product images.

- ✓ Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate to deploy a containerized application. Use Amazon RDS with a Multi-AZ deployment to store the product data. Store the product images in an Amazon S3 bucket.

This option uses ECS with Fargate, providing a serverless container orchestration solution that minimizes operational overhead. RDS with Multi-AZ ensures high availability and data persistence while maintaining the existing structured database schemas. S3 is ideal for storing and serving product images efficiently.

Overall Explanation:

The best solution is to use Amazon ECS with Fargate, Amazon RDS with a Multi-AZ deployment, and Amazon S3. ECS Fargate provides a serverless approach to container management, minimizing operational overhead. RDS maintains structured database schemas, and Multi-AZ ensures high availability. S3 provides scalable and cost-effective storage for product images. The other options either increase operational overhead, don't maintain structured schemas, or are not optimized for frequently accessed product images.

[View Discussions](#)[Ask AI](#)

Question 7

Not Attempted

Security



A company is migrating an application from an on-premises environment to AWS. The application will store sensitive data in Amazon S3. The company's security policy mandates that all data must be encrypted **before** being sent to Amazon S3 for storage. Which solution will meet these requirements?

- ☒ Encrypt the data by using client-side encryption with customer-managed keys.
This option meets the requirement as the data is encrypted before being sent to S3, and the keys are managed by the customer, offering the highest level of control.
- ☐ Encrypt the data by using server-side encryption with AWS KMS keys (SSE-KMS).
SSE-KMS encrypts the data at rest in S3, but the data is not encrypted before being sent to S3.
- ☐ Encrypt the data by using server-side encryption with customer-provided keys (SSE-C).
SSE-C encrypts the data at rest in S3, but the data is not encrypted before being sent to S3. Also, it requires the customer to manage the keys, but the encryption is done server-side.
- ☐ Encrypt the data by using client-side encryption with Amazon S3-managed keys.
Client-side encryption with S3-managed keys is not a valid option. If using client-side encryption, the customer manages the keys.

Overall Explanation:

The requirement is to encrypt the data **before** storing it in Amazon S3. Therefore, server-side encryption options (SSE-KMS and SSE-C) are not suitable because they encrypt the data at rest in S3, not before. Client-side encryption with customer-managed keys is the correct solution as it encrypts the data locally before sending it to S3, fulfilling the requirement. Client-side encryption with Amazon S3-managed keys is not a valid option.

View Discussions

Ask AI

Question 8

Not Attempted

Security



A company wants to create an Amazon EMR cluster that will be used by multiple teams. The company requires that each team's big data workloads have access only to the specific AWS services each team needs. The company also needs to prevent the workloads from accessing

Instance Metadata Service Version 2 (IMDSv2) on the cluster's underlying EC2 instances. Which solution meets these requirements?

- ⊗ Configure interface VPC endpoints for each AWS service that the teams need. Use the required interface VPC endpoints to submit the big data workloads.

While VPC endpoints enhance security by keeping traffic within the AWS network, they don't address the granular permission requirements for each team or prevent access to IMDSv2.

- ✓ Create EMR runtime roles. Configure the cluster to use the runtime roles. Use the runtime roles to submit the big data workloads.

This is the correct solution. EMR runtime roles allow you to grant specific permissions to each team's workloads, ensuring they only access the necessary AWS services. Furthermore, runtime roles avoid reliance on the EC2 instance profile, minimizing the need to access IMDSv2.

- ⊗ Create an EC2 IAM instance profile that has the required permissions for each team. Use the instance profile to submit the big data workloads.

EC2 IAM instance profiles are applied at the instance level, meaning all workloads running on the same instance would inherit the same permissions, which doesn't meet the requirement of granting different permissions to different teams.

- ⊗ Create an EMR security configuration that has the `EnableApplicationScopedIAMRole` option set to false. Use the security configuration to submit the big data workloads.

Disabling `EnableApplicationScopedIAMRole` in the EMR security configuration doesn't provide granular permissions control for each team's workloads or prevent access to IMDSv2. It primarily affects how IAM roles are handled within applications.

Overall Explanation:

EMR runtime roles enable you to assign granular permissions to individual EMR jobs or steps, ensuring each team's workloads can only access the AWS services they need. This approach also avoids the need for workloads to access the Instance Metadata Service (IMDSv2) on the underlying EC2 instances, as permissions are managed through the runtime roles.

 View Discussions

 Ask AI

Question 9

Not Attempted

Compute



A solutions architect is designing an application to process registration forms. The application uses a two-tier architecture with a web application server tier and a worker tier. Submitted forms need to be processed quickly and exactly once, ensuring no data loss. Which solution meets these requirements?

- ☒ Use an Amazon Simple Queue Service (Amazon SQS) FIFO queue between the web application server tier and the worker tier to store and forward form data.
SQS FIFO queues guarantee first-in, first-out delivery and exactly-once processing, preventing duplicates and ensuring reliable data handling. This meets the requirements for speed and processing each form only once.
- ☐ Use an Amazon API Gateway HTTP API between the web application server tier and the worker tier to store and forward form data.
API Gateway doesn't inherently guarantee exactly-once processing or queuing, making it unsuitable for ensuring no data loss and single processing.
- ☐ Use an Amazon Simple Queue Service (Amazon SQS) standard queue between the web application server tier and the worker tier to store and forward form data.
SQS standard queues provide best-effort ordering and can deliver messages more than once, failing to meet the 'exactly once' requirement.
- ☐ Use an AWS Step Functions workflow. Create a synchronous workflow between the web application server tier and the worker tier that stores and forwards form data.
While Step Functions can manage workflows, a synchronous workflow might introduce latency and doesn't inherently provide the queuing and exactly-once processing guarantees of SQS FIFO.

Overall Explanation:

SQS FIFO queues are designed for scenarios requiring ordered and exactly-once message processing. By using an SQS FIFO queue, the application can reliably forward form data from the web application tier to the worker tier, ensuring that each form is processed quickly and exactly once without data loss. Standard queues do not guarantee order or exactly-once processing, and API Gateway and synchronous Step Functions workflows don't offer the same level of inherent reliability and queuing.

[View Discussions](#)[Ask AI](#)

A finance company uses an on-premises search application to collect streaming data from various producers. The application provides real-time updates to search and visualization features. The company is planning to migrate to AWS and wants to use an AWS native solution that provides similar real-time search and visualization capabilities. Which solution will best meet these requirements?

- ☐ Use Amazon EC2 instances to ingest and process the data streams to Amazon S3 buckets for storage. Use Amazon Athena to search the data. Use Amazon Managed Grafana to create visualizations.
This solution uses S3 for storage and Athena for querying which is not optimized for real-time updates and search.
- ☐ Use Amazon EMR to ingest and process the data streams to Amazon Redshift for storage. Use Amazon Redshift Spectrum to search the data. Use Amazon QuickSight to create visualizations.
This solution is not optimal for real-time data ingestion. Redshift is primarily a data warehouse and not intended for handling streaming data directly.
- ☐ Use Amazon Elastic Kubernetes Service (Amazon EKS) to ingest and process the data streams to Amazon DynamoDB for storage. Use Amazon CloudWatch to create graphical dashboards to search and visualize the data.
While DynamoDB is a good choice for some workloads, CloudWatch dashboards are not designed for the kind of interactive search and visualization the company requires. EKS adds unnecessary complexity.
- ☒ Use Amazon Kinesis Data Streams to ingest and process the data streams to Amazon OpenSearch Service. Use OpenSearch Service to search the data. Use Amazon QuickSight to create visualizations.
This solution uses Kinesis Data Streams for real-time data ingestion, OpenSearch Service for fast search, and QuickSight for visualizations, addressing the company's requirements effectively.

Overall Explanation:

The best solution is to use Amazon Kinesis Data Streams, Amazon OpenSearch Service, and Amazon QuickSight. Kinesis Data Streams is designed for real-time data ingestion, OpenSearch Service offers powerful search capabilities and QuickSight enables data visualization.

[View Discussions](#)[Ask AI](#)

Question 11

Not Attempted

[Compute](#)

A company runs an on-premises ASP.NET application on Linux servers, serving customers directly. The application is resource-intensive, and the company wants to modernize it to .NET and run it in containers. They need to scale the application based on Amazon CloudWatch metrics and minimize operational overhead. Which solution best meets these requirements with the LEAST operational overhead?

- ☒ Use AWS App2Container to containerize the application. Use an AWS CloudFormation template to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.
App2Container simplifies containerizing existing applications. Deploying to ECS on Fargate eliminates the need to manage EC2 instances, reducing operational overhead. Fargate also integrates with CloudWatch metrics for scaling.
- ☐ Use AWS App2Container to containerize the application. Use an AWS CloudFormation template to deploy the application to Amazon Elastic Container Service (Amazon ECS) on Amazon EC2 instances.
While App2Container helps with containerization, deploying to ECS on EC2 instances requires managing the underlying infrastructure, increasing operational overhead.
- ☐ Use AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Container Service (Amazon ECS) on AWS Fargate.
AWS App Runner is a fully managed service for deploying containerized applications directly. It does not deploy applications *to* ECS or EKS.
- ☐ Use AWS App Runner to containerize the application. Use App Runner to deploy the application to Amazon Elastic Kubernetes Service (Amazon EKS) on Amazon EC2 instances.
AWS App Runner is a fully managed service for deploying containerized applications directly. It does not deploy applications *to* ECS or EKS.

Overall Explanation:

The optimal solution is to use AWS App2Container to containerize the ASP.NET application and then deploy it to Amazon ECS on AWS Fargate using a CloudFormation template. App2Container automates the containerization process, and Fargate eliminates the need to

manage the underlying EC2 instances, drastically reducing operational overhead. Fargate also integrates with Amazon CloudWatch to facilitate scaling based on custom metrics. AWS App Runner does not deploy *to* ECS or EKS.

[View Discussions](#)[Ask AI](#)

Question 12

Not Attempted

Storage



A company's on-premises Microsoft SharePoint deployment utilizes Microsoft Windows shared file storage. The company intends to migrate this workload to the AWS Cloud. The chosen storage solution must be highly available and integrate with Active Directory for access control. Which solution meets these requirements?

☐ Configure Amazon EFS storage and set the Active Directory domain for authentication. EFS does not provide native integration with Active Directory for access control. While it can be configured, it requires additional setup and is not as straightforward as using FSx for Windows File Server.

☐ Create an SMB file share on an AWS Storage Gateway file gateway in two Availability Zones. Using Storage Gateway introduces additional complexity in this scenario, requiring maintenance of the gateway and synchronization management between on-premises and AWS storage.

☐ Create an Amazon S3 bucket and configure Microsoft Windows Server to mount it as a volume. S3 does not natively support the SMB file protocol required for Microsoft SharePoint and Windows shared file storage. Mounting S3 as a volume requires third-party tools or configurations and is not recommended.

☒ Create an Amazon FSx for Windows File Server file system on AWS and set the Active Directory domain for authentication. FSx for Windows File Server is a fully managed, highly available file storage service compatible with Windows shared file storage requirements. It provides native integration with Active Directory for seamless access control using existing AD user accounts.

Overall Explanation:

Amazon FSx for Windows File Server is the most suitable solution. It's a fully managed file storage service designed for Microsoft Windows workloads, offering high availability and

native Active Directory integration for access control. It avoids the complexities of EFS or Storage Gateway and directly addresses the need for Windows-compatible shared file storage.

[View Discussions](#)[Ask AI](#)

Question 13

Not Attempted

Security



A company is designing a new internal web application in the AWS Cloud. The new application must securely retrieve and store multiple employee usernames and passwords from an AWS managed service. The application will use CloudFormation for infrastructure as code and requires the ability to retrieve multiple secrets in a single API call. Which solution will meet these requirements with the LEAST operational overhead?

- ☐ Store the employee credentials in AWS Systems Manager Parameter Store. Use AWS CloudFormation and the BatchGetSecretValue API to retrieve usernames and passwords from Parameter Store.

Systems Manager Parameter Store is suitable for configuration data, but Secrets Manager is preferred for sensitive credentials. Parameter Store also doesn't integrate with the BatchGetSecretValue API.

- ☐ Store the employee credentials in AWS Secrets Manager. Use AWS CloudFormation and AWS Batch with the BatchGetSecretValue API to retrieve the usernames and passwords from Secrets Manager.

While Secrets Manager is a good choice, introducing AWS Batch for retrieving secrets adds unnecessary complexity and operational overhead. AWS Batch is for batch processing, not secret retrieval.

- ☐ Store the employee credentials in AWS Systems Manager Parameter Store. Use AWS CloudFormation and AWS Batch with the BatchGetSecretValue API to retrieve the usernames and passwords from Parameter Store.

Systems Manager Parameter Store is suitable for configuration data, but Secrets Manager is preferred for sensitive credentials. Also, using AWS Batch adds unnecessary overhead.

- ☒ Store the employee credentials in AWS Secrets Manager. Use AWS CloudFormation and the BatchGetSecretValue API to retrieve the usernames and passwords from Secrets Manager.

This option provides a secure and appropriate solution using AWS Secrets Manager, integrates with CloudFormation, and utilizes the BatchGetSecretValue API for efficient secret retrieval with the least operational overhead.

Overall Explanation:

The optimal solution is to store employee credentials in AWS Secrets Manager and use CloudFormation in conjunction with the BatchGetSecretValue API. Secrets Manager is designed for secure storage and retrieval of secrets. The BatchGetSecretValue API allows efficient retrieval of multiple secrets in a single API call, minimizing overhead. Using AWS Batch is not necessary and adds unnecessary complexity.

[View Discussions](#)[Ask AI](#)

Question 14

Not Attempted

Networking



A company operating in the ap-northeast-1 Region manages thousands of AWS Outposts servers deployed globally at remote locations. These servers regularly download new software versions, each consisting of 100 files. There is significant latency before all servers run the updated software. The company needs to minimize deployment latency for new software versions with the LEAST operational overhead. Which solution meets these requirements?

- ⊗ Create an Amazon S3 bucket in ap-northeast-1. Set up an Amazon CloudFront distribution in ap-northeast-1 that includes a CachingDisabled cache policy. Configure the S3 bucket as the origin. Download the software by using signed URLs.

Disabling caching in CloudFront increases latency and defeats the purpose of using a CDN. It ensures that the freshest content is always delivered, but it does so at the cost of performance, which contradicts the need to reduce deployment latency.

- ⊗ Create an Amazon S3 bucket in ap-northeast-1. Create a second S3 bucket in the us-east-1 Region. Configure replication between the buckets. Set up an Amazon CloudFront distribution that uses ap-northeast-1 as the primary origin and us-east-1 as the secondary origin. Download the software by using signed URLs.

Using dual-region origins with CloudFront adds unnecessary complexity without providing a significant latency reduction compared to using a single origin with CloudFront's edge caching. This introduces additional operational overhead and costs.

- ⊗ Create an Amazon S3 bucket in ap-northeast-1. Configure Amazon S3 Transfer Acceleration. Download the software by using the S3 Transfer Acceleration endpoint.

S3 Transfer Acceleration is more focused on accelerating data transfer to and from S3 buckets, but it does not offer the same global caching benefits as CloudFront. While it can improve download speeds, it doesn't distribute the software to edge locations closer to the servers like CloudFront does. However, using S3 Transfer Acceleration provides a simpler setup.

- ✔ Create an Amazon S3 bucket in ap-northeast-1. Set up an Amazon CloudFront distribution. Configure the S3 bucket as the origin. Download the software by using signed URLs.

Setting up a CloudFront distribution with the S3 bucket as the origin caches the software versions at edge locations close to the remote AWS Outposts servers. This reduces download latency because servers access the content from the nearest CloudFront edge location instead of downloading directly from S3 in ap-northeast-1. It balances effectiveness and operational overhead well.

Overall Explanation:

CloudFront delivers content from edge locations closer to users (in this case, the Outposts servers), significantly reducing latency compared to accessing S3 directly from a single region. S3 Transfer Acceleration is designed for uploads and downloads to S3 and does not provide the content distribution and caching capabilities of CloudFront. A CloudFront configuration with CachingDisabled unnecessarily increases latency. Multi-region origins introduce unnecessary complexity.

 View Discussions

 Ask AI

Question 15

Not Attempted

Storage



A company currently runs an on-premises stock trading application by using Microsoft Windows Server. The company wants to migrate the application to the AWS Cloud. The company needs to design a highly available solution that provides low-latency access to block storage across multiple Availability Zones. Which solution will meet these requirements with the LEAST implementation effort?

- ✘ Configure a Windows Server cluster that spans two Availability Zones on Amazon EC2 instances. Install the application on both cluster nodes. Use Amazon FSx for Windows File Server as shared storage between the two cluster nodes.

FSx for Windows File Server is not designed for block storage. It's optimized for shared file storage, making it unsuitable for low-latency block storage requirements.

- ⊗ Configure a Windows Server cluster that spans two Availability Zones on Amazon EC2 instances. Install the application on both cluster nodes. Use Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp3) volumes as storage attached to the EC2 instances. Set up application-level replication to sync data from one EBS volume in one Availability Zone to another EBS volume in the second Availability Zone.

Setting up application-level replication adds significant complexity and implementation effort. This approach is not the least effort solution.

- ✓ Deploy the application on Amazon EC2 instances in two Availability Zones. Configure one EC2 instance as active and the second EC2 instance in standby mode. Use an Amazon FSx for NetApp ONTAP Multi-AZ file system to access the data by using Internet Small Computer Systems Interface (iSCSI) protocol.

Amazon FSx for NetApp ONTAP supports block storage (via iSCSI) and provides Multi-AZ fault tolerance out of the box, fulfilling the requirements with less implementation effort.

- ⊗ Deploy the application on Amazon EC2 instances in two Availability Zones. Configure one EC2 instance as active and the second EC2 instance in standby mode. Use Amazon Elastic Block Store (Amazon EBS) Provisioned IOPS SSD (io2) volumes as storage attached to the EC2 instances. Set up Amazon EBS level replication to sync data from one io2 volume in one Availability Zone to another io2 volume in the second Availability Zone.

EBS io2 volumes with replication across AZs are complex to configure and manage, making this option more effort-intensive than using a managed service like FSx for NetApp ONTAP.

Overall Explanation:

The company requires low-latency access to block storage. Amazon FSx for NetApp ONTAP provides a managed file storage solution that supports the iSCSI protocol for block storage access, enabling low-latency, highly available access to shared storage across multiple Availability Zones. This solution requires less implementation effort because Amazon FSx manages the complex underlying multi-AZ configuration and data replication automatically.

 View Discussions

 Ask AI

Question 16

Not Attempted

Networking



A company is designing a web application with an internet-facing Application Load Balancer (ALB). The company needs the ALB to receive HTTPS web traffic from the public internet. The ALB must send only HTTPS traffic to the web application servers hosted on Amazon EC2 instances on port 443. The ALB must perform a health check of the web application servers over HTTPS on port 8443. Which combination of configurations of the security group that is associated with the ALB will meet these requirements?

- ☒ Allow HTTPS inbound traffic from 0.0.0.0/0 for port 443.
This allows the ALB to receive HTTPS traffic from the public internet on port 443.
- ☐ Allow all outbound traffic to 0.0.0.0/0 for port 443.
This option allows outbound traffic on port 443 to any destination, which is broader than necessary and potentially less secure.
- ☒ Allow HTTPS outbound traffic to the web application instances for port 443.
This allows the ALB to forward HTTPS traffic to the web application servers on port 443.
- ☐ Allow HTTPS inbound traffic from the web application instances for port 443.
This is not required, as the ALB is initiating outbound traffic to the web servers, not the other way around.
- ☒ Allow HTTPS outbound traffic to the web application instances for the health check on port 8443.
This allows the ALB to perform health checks on the web application servers over HTTPS on port 8443.
- ☐ Allow HTTPS inbound traffic from the web application instances for the health check on port 8443.
This is not required as the ALB is initiating outbound traffic for the health checks.

Overall Explanation:

The ALB needs to receive inbound HTTPS traffic from the internet (0.0.0.0/0) on port 443. It also needs to send outbound HTTPS traffic to the web application instances on port 443 for normal operation and on port 8443 for health checks. Therefore, options A, C, and E are the correct configurations for the ALB's security group.

[View Discussions](#)[Ask AI](#)

Question 17

Not Attempted

Networking



A company hosts an application on AWS. The application allows users to upload photos and store them in an Amazon S3 bucket. The company wants to use Amazon CloudFront and a custom domain name to upload the photo files to the S3 bucket in the eu-west-1 Region. Which two solutions will meet these requirements?

- ☒ Use AWS Certificate Manager (ACM) to create a public certificate in the us-east-1 Region. Use the certificate in CloudFront.
CloudFront requires SSL/TLS certificates for custom domains to be provisioned in the us-east-1 Region, regardless of the origin's location.
- ☐ Use AWS Certificate Manager (ACM) to create a public certificate in eu-west-1. Use the certificate in CloudFront.
CloudFront only supports certificates in the us-east-1 Region for use with custom domain names.
- ☐ Configure Amazon S3 to allow uploads from CloudFront. Configure S3 Transfer Acceleration.
S3 Transfer Acceleration speeds up transfers over long distances, but doesn't secure or restrict access for uploads from CloudFront.
- ☒ Configure Amazon S3 to allow uploads from CloudFront origin access control (OAC).
Configuring S3 to allow uploads from CloudFront using Origin Access Control (OAC) ensures that only CloudFront can interact with the S3 bucket, improving security by preventing direct public access.
- ☐ Configure Amazon S3 to allow uploads from CloudFront. Configure an Amazon S3 website endpoint.
Configuring an S3 website endpoint is not needed for uploads from CloudFront and doesn't provide specific access control for CloudFront.

Overall Explanation:

The correct answers are A and D. CloudFront requires the SSL/TLS certificate for the custom domain to be created in the us-east-1 Region (N. Virginia). Using Origin Access

Control (OAC) ensures secure uploads from CloudFront to the S3 bucket by preventing direct public access.

[View Discussions](#)[Ask AI](#)

Question 18

Not Attempted

Cost Optimization



A weather forecasting company collects temperature readings from various sensors on a continuous basis. An existing data ingestion process collects the readings and aggregates the readings into larger Apache Parquet files. The process encrypts the files by using client-side encryption with KMS managed keys (CSE-KMS). Finally, the process writes the files to an Amazon S3 bucket with separate prefixes for each calendar day. The company wants to run occasional SQL queries on the data to take sample moving averages for a specific calendar day. Which solution will meet these requirements MOST cost-effectively?

- ☒ **Configure Amazon Athena to read the encrypted files. Run SQL queries on the data directly in Amazon S3.**
Athena is serverless, works with Parquet, supports encryption, and is cost-effective as you pay only for queried data. It can handle running SQL queries and decrypting the files, making it a suitable option.
- ☐ **Use Amazon S3 Select to run SQL queries on the data directly in Amazon S3.**
S3 Select is suitable for simple filtering but might not be the best option for more complex queries like calculating moving averages. It might not natively support CSE-KMS decryption.
- ☐ **Configure Amazon Redshift to read the encrypted files. Use Redshift Spectrum and Redshift query editor v2 to run SQL queries on the data directly in Amazon S3.**
Redshift Spectrum is powerful but overkill for occasional SQL queries. Setting up and maintaining Redshift is also more complex and costly.
- ☐ **Configure Amazon EMR Serverless to read the encrypted files. Use Apache SparkSQL to run SQL queries on the data directly in Amazon S3.**
EMR Serverless is designed for heavy data pipelines, making it too complex and expensive for occasional SQL analysis.

Overall Explanation:

Amazon Athena is the most cost-effective solution. It is a serverless query service that can directly query data in Amazon S3, supports client-side encryption with KMS keys, and only charges for the queries that are run. Redshift Spectrum and EMR Serverless are more complex and expensive solutions that are not necessary for this use case. S3 Select is not ideal for complex queries like moving averages.

[View Discussions](#)[Ask AI](#)

Question 19

Not Attempted

Networking



A company is deploying a new application on AWS, utilizing multiple Amazon EC2 instances across various Availability Zones and AWS Regions. The application will be accessible globally via the internet. The company requires that users are directed to the EC2 instances that offer the lowest latency based on the user's location. Which solution best satisfies these requirements?

- ☐ Implement an Amazon Route 53 geolocation routing policy. Use an internet-facing Application Load Balancer to distribute the traffic across all Availability Zones within the same Region.

Geolocation routing directs traffic based on the user's geographic location. However, it may not always route to the physically closest instances based on network latency. An Application Load Balancer is suitable for HTTP/HTTPS traffic but might not be optimal for all application types.

- ☒ Implement an Amazon Route 53 geoproximity routing policy. Use an internet-facing Network Load Balancer to distribute the traffic across all Availability Zones within the same Region.

Geoproximity routing considers both the user's location and the resource's location, routing based on proximity. This ensures users are routed to the closest instances based on network latency, which fits the prompt's requirements. A Network Load Balancer is suitable for high-performance applications with TCP/UDP traffic.

- ☐ Implement an Amazon Route 53 multivalue answer routing policy. Use an internet-facing Application Load Balancer to distribute the traffic across all Availability Zones within the same Region.

Multivalue answer routing returns multiple healthy resources to the user, but does not take user location into account.

- ⊗ Implement an Amazon Route 53 weighted routing policy. Use an internet-facing Network Load Balancer to distribute the traffic across all Availability Zones within the same Region.

Weighted routing distributes traffic based on weights assigned to each resource. This does not consider user location, and therefore it does not satisfy the primary requirement.

Overall Explanation:

The best solution is to use Amazon Route 53 geoproximity routing, which takes into account both the user's and the resources' location to route traffic to the nearest resources. A Network Load Balancer can provide high performance and low latency. Geoproximity considers both the user's and resource's geographic locations to make a routing decision based on latency. Geolocation only uses the user's location.

 View Discussions

 Ask AI

Question 20

Not Attempted

Security



A financial services company plans to launch a new application on AWS to handle sensitive financial transactions. The company will deploy the application on Amazon EC2 instances and use Amazon RDS for MySQL as the database. The company's security policies mandate that data must be encrypted at rest and in transit. Which solution will meet these requirements with the LEAST operational overhead?

- ✔ Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure AWS Certificate Manager (ACM) SSL/TLS certificates for encryption in transit.

This option provides encryption at rest using KMS managed keys, which simplifies key management. ACM handles SSL/TLS certificates for encryption in transit, automating certificate renewal and deployment and minimizing operational overhead.

- ⊗ Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure IPsec tunnels for encryption in transit.

While IPsec tunnels provide encryption in transit, they involve more configuration and maintenance than ACM SSL/TLS certificates, increasing operational overhead.

- ⊗ Implement third-party application-level data encryption before storing data in Amazon RDS for MySQL. Configure AWS Certificate Manager (ACM) SSL/TLS certificates for

encryption in transit.

Implementing third-party application-level encryption adds complexity and operational overhead compared to using native RDS encryption at rest with KMS. ACM for encryption in transit is a good approach.

- ⊗ Configure encryption at rest for Amazon RDS for MySQL by using AWS KMS managed keys. Configure a VPN connection to enable private connectivity to encrypt data in transit.

Configuring a VPN connection for encryption in transit introduces significantly more overhead than using ACM. While VPNs offer private connectivity, ACM provides a simpler solution for encrypting data in transit to RDS.

Overall Explanation:

Option A offers the least operational overhead by utilizing AWS KMS for encryption at rest and AWS Certificate Manager (ACM) for encryption in transit. KMS managed keys simplify key management, and ACM automates certificate management, thus reducing the overall administrative burden. Options B, C, and D involve more complex configurations or manual steps, leading to increased operational overhead.

 View Discussions

 Ask AI

Question 21

Not Attempted

Databases



A company is migrating its on-premises Oracle database to an Amazon RDS for Oracle database. The company needs to retain data for 90 days to meet regulatory requirements. The company must also be able to restore the database to a specific point in time for up to 14 days. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create Amazon RDS automated backups. Set the retention period to 90 days.
Amazon RDS automated backups support a maximum retention period of 35 days. This option does not meet the requirement to retain backups for 90 days.
- ⊗ Create an Amazon RDS manual snapshot every day. Delete manual snapshots that are older than 90 days.
This approach requires manual snapshot management, including scheduling snapshots and deleting old ones. This increases operational overhead and is prone to human error.

- ☐ Use the Amazon Aurora Clone feature for Oracle to create a point-in-time restore. Delete clones that are older than 90 days.

This option is not applicable as Aurora Clone is a feature specific to Amazon Aurora and not available for Amazon RDS for Oracle. Additionally, it would require manual management of clones, increasing complexity.

- ☒ Create a backup plan that has a retention period of 90 days by using AWS Backup for Amazon RDS.

AWS Backup supports point-in-time recovery for Amazon RDS, enabling you to restore the database to any specific point within the defined retention period, up to 35 days. For the requirement of 14 days, AWS Backup easily supports this capability and fulfills the requirement of 90 days retention.

Overall Explanation:

AWS Backup for Amazon RDS provides a centralized and automated way to manage backups and retention policies, minimizing operational overhead while meeting the 90-day retention and point-in-time recovery requirements. Automated backups only support a maximum of 35 days retention. Manual snapshots involve manual management. Aurora Clone is not available for RDS Oracle.

 View Discussions

 Ask AI

Question 22

Not Attempted

Databases



A company is developing a read-heavy application using a relational database to store user data and application configurations. They anticipate steady user growth and variable database usage patterns, with infrequent write operations. To optimize costs, the company seeks an AWS managed database solution that provides the required performance. Which solution is MOST cost-effective for this scenario?

- ☐ Deploy the database on Amazon RDS using Provisioned IOPS SSD storage to ensure consistent performance for read and write operations.

Provisioned IOPS SSD storage incurs costs even during periods of low activity, making it less cost-effective for variable workloads.

- ☒ Deploy the database on Amazon Aurora Serverless to automatically scale database capacity based on actual usage to accommodate the workload.

Aurora Serverless automatically scales based on demand, minimizing costs during low-usage periods, making it ideal for variable and read-heavy workloads.

- ⊗ Deploy the database on Amazon DynamoDB. Use on-demand capacity mode to automatically scale throughput to accommodate the workload.

DynamoDB is a NoSQL database and not a suitable choice for relational data. Also, switching from relational database to DynamoDB is a massive change.

- ⊗ Deploy the database on Amazon RDS. Use magnetic storage and use read replicas to accommodate the workload.

Magnetic storage offers significantly slower IOPS compared to SSDs, potentially impacting performance. Also, Aurora uses SSD storage.

Overall Explanation:

Aurora Serverless automatically scales database capacity based on the application's actual usage, resulting in optimal cost savings for variable, read-heavy workloads with occasional writes. Provisioned IOPS would be more expensive than Aurora Serverless. DynamoDB is not a suitable solution since the company wants to utilize a relational database.

[View Discussions](#)[Ask AI](#)

Question 23

Not Attempted

Serverless



An image-processing company has a web application that users use to upload images. The application uploads the images into an Amazon S3 bucket. The company has set up S3 event notifications to publish the object creation events to an Amazon Simple Queue Service (Amazon SQS) standard queue. The SQS queue serves as the event source for an AWS Lambda function that processes the images and sends the results to users through email. Users report that they are receiving multiple email messages for every uploaded image. A solutions architect determines that SQS messages are invoking the Lambda function more than once, resulting in multiple email messages. What should the solutions architect do to resolve this issue with the LEAST operational overhead?

- ⊗ Set up long polling in the SQS queue by increasing the ReceiveMessage wait time to 30 seconds.

Long polling doesn't directly address the issue of duplicate invocations. Increasing the ReceiveMessage wait time might reduce the frequency of polls, but it doesn't prevent duplicate

processing if the visibility timeout expires before the Lambda function completes.

- ⊗ Change the SQS standard queue to an SQS FIFO queue. Use the message deduplication ID to discard duplicate messages.

While FIFO queues prevent duplicates, changing a standard queue to a FIFO queue involves significant architectural changes and operational overhead. It requires modifying event configurations, handling message deduplication IDs, and ensuring the application is compatible with FIFO queue semantics.

- ✔ Increase the visibility timeout in the SQS queue to a value that is greater than the total of the function timeout and the batch window timeout.

Increasing the visibility timeout ensures that the message is not made visible to other consumers while the Lambda function is processing it. This prevents duplicate invocations and has the least operational overhead, as it involves a simple configuration change to the SQS queue.

- ⊗ Modify the Lambda function to delete each message from the SQS queue immediately after the message is read before processing.

Deleting the message immediately after reading it can lead to message loss if the Lambda function encounters an error during processing. It does not guarantee successful message processing and can result in data loss.

Overall Explanation:

The issue is that the Lambda function is being invoked multiple times for the same message because the visibility timeout on the SQS queue is too short. Before the first Lambda function finishes processing the message and deleting it from the queue, the message becomes visible again, and another Lambda function picks it up. Increasing the visibility timeout to be longer than the Lambda function's execution time prevents the message from becoming visible again prematurely. This resolves the duplicate message processing issue with the least amount of operational overhead. Therefore, increasing the visibility timeout in the SQS queue to a value that is greater than the total of the function timeout and the batch window timeout is the best solution.

[View Discussions](#)[Ask AI](#)

Question 24

Not Attempted

Security



A company hosts its application on several Amazon EC2 instances inside a VPC. The company creates a dedicated Amazon S3 bucket for each customer to store their relevant information in Amazon S3. The company wants to ensure that the application running on EC2 instances can securely access only the S3 buckets that belong to the company's AWS account. Which solution will meet these requirements with the LEAST operational overhead?

- ⊗ Create a gateway endpoint for Amazon S3 that is attached to the VPC. Update the IAM instance profile policy to provide access to only the specific buckets that the application needs.

This option leverages a gateway endpoint for S3 access, which is efficient. Updating the IAM role to explicitly allow access to each company-owned bucket provides security, but could become complex to manage as the number of buckets grows.

- ⊗ Create a NAT gateway in a public subnet with a security group that allows access to only Amazon S3. Update the route tables to use the NAT Gateway.

A NAT gateway is designed for internet-bound traffic and is not the most efficient or secure method for accessing S3 within the same AWS region. It also introduces more operational overhead than other options.

- ✔ Create a gateway endpoint for Amazon S3 that is attached to the VPC. Update the IAM instance profile policy with a Deny action and the following condition key:

This option correctly utilizes a Gateway Endpoint for secure and efficient access. It further enhances security by using a Deny action with a condition key in the IAM policy to prevent access to S3 buckets outside the company's AWS account.

- ⊗ Create a NAT Gateway in a public subnet. Update route tables to use the NAT Gateway. Assign bucket policies for all buckets with a Deny action and the following condition key:

Using a NAT gateway is not optimal for accessing S3 buckets within the same AWS region. Additionally, managing bucket policies for each bucket introduces significant operational overhead.

Overall Explanation:

Option C is the best solution. It uses a gateway endpoint for secure and efficient S3 access, minimizing operational overhead. The IAM instance profile policy with a Deny action and the condition key restricts access to only the company's S3 buckets. Options B and D involve using a NAT Gateway, which is not ideal for S3 access within the same region. Option A requires managing explicit allow rules for each bucket, increasing operational overhead as the number of buckets grows.

[View Discussions](#)[Ask AI](#)

Question 25

Not Attempted

Security



A company is building a cloud-based application on AWS that handles sensitive customer data. The application uses Amazon RDS for the database, Amazon S3 for object storage, and S3 Event Notifications that invoke AWS Lambda for serverless processing. The company uses AWS IAM Identity Center to manage user credentials. The development, testing, and operations teams need secure access to Amazon RDS and Amazon S3 while ensuring the confidentiality of sensitive customer data. The solution must comply with the principle of least privilege. Which solution meets these requirements with the LEAST operational overhead?

- ☐ Use IAM roles with least privilege to grant all the teams access. Assign IAM roles to each team with customized IAM policies defining specific permission for Amazon RDS and S3 object access based on team responsibilities.
Managing and customizing IAM roles for multiple teams can become complex and high-maintenance, especially as team responsibilities evolve, leading to higher operational overhead.
- ☒ Enable IAM Identity Center with an Identity Center directory. Create and configure permission sets with granular access to Amazon RDS and Amazon S3. Assign all the teams to groups that have specific access with the permission sets.
IAM Identity Center allows you to manage and update access for multiple teams from one place using permission sets and group assignments. This approach minimizes operational overhead while maintaining secure and compliant access.
- ☐ Create individual IAM users for each member in all the teams with role-based permissions. Assign the IAM roles with predefined policies for RDS and S3 access to each user based on user needs. Implement IAM Access Analyzer for periodic credential evaluation.
Creating individual IAM users is labor-intensive and adds operational overhead.
- ☐ Use AWS Organizations to create separate accounts for each team. Implement cross-account IAM roles with least privilege. Grant specific permission for RDS and S3 access based on team roles and responsibilities.
Cross-account access is harder to manage and introduces additional operational overhead compared to using IAM Identity Center.

Overall Explanation:

IAM Identity Center, with its centralized user management, permission sets, and group assignments, offers the least operational overhead for managing access across multiple teams while adhering to the principle of least privilege. It simplifies access management by enabling you to define and update access for multiple teams from a single location. While using IAM roles can work, it becomes more complex and high-maintenance as teams and responsibilities grow. Creating individual IAM users for each team member is also operationally intensive. Managing cross-account access is harder to manage and introduces additional operational overhead.

[View Discussions](#)[Ask AI](#)

Question 26

Not Attempted

Security



A company has an Amazon S3 bucket containing sensitive data files. An application running on on-premises virtual machines requires temporary access to these files. The company uses AWS IAM Identity Center. What is the MOST secure method to grant the application access to the S3 bucket?

- ☐ Create an S3 bucket policy that permits access to the bucket from the public IP address range of the company's on-premises data center.
This option is not recommended as it grants access based on IP address, which can be spoofed and is not a secure practice.
- ☒ Use IAM Roles Anywhere to obtain security credentials in IAM Identity Center that grant access to the S3 bucket. Configure the virtual machines to assume the role by using the AWS CLI.
This is the MOST secure option. IAM Roles Anywhere allows on-premises applications to assume IAM roles, granting temporary credentials without storing long-term access keys. It integrates well with IAM Identity Center.
- ☐ Install the AWS CLI on the virtual machine. Configure the AWS CLI with access keys from an IAM user that has access to the bucket.
This option involves storing long-term access keys on the virtual machines, which is a security risk if the virtual machines are compromised.
- ☐ Create an IAM user and policy that grants access to the bucket. Store the access key and secret key for the IAM user in AWS Secrets Manager. Configure the application to

retrieve the access key and secret key at startup.

While using Secrets Manager is better than storing credentials directly, it still involves managing long-term credentials. IAM Roles Anywhere is a more secure and preferred approach.

Overall Explanation:

IAM Roles Anywhere provides a secure method to grant temporary access to AWS resources for applications running outside of AWS. By utilizing IAM roles, the application can assume temporary credentials without the need for long-term access keys. This method integrates with AWS IAM Identity Center and aligns with security best practices.

[View Discussions](#)[Ask AI](#)

Question 27

Not Attempted

Networking



A company hosts its core network services, including directory services and DNS, in its on-premises data center. The data center is connected to the AWS Cloud using AWS Direct Connect (DX). Additional AWS accounts are planned that will require quick, cost-effective, and consistent access to these network services. What should a solutions architect implement to meet these requirements with the LEAST amount of operational overhead?

- ☐ Create a DX connection in each new account. Route the network traffic to the on-premises servers.

Managing separate DX connections for every account is too much work, and DX connections are expensive.

- ☐ Configure VPC endpoints in the DX VPC for all required services. Route the network traffic to the on-premises servers.

VPC endpoints are for accessing AWS services, not on-premises services.

- ☐ Create a VPN connection between each new account and the DX VPC. Route the network traffic to the on-premises servers.

Managing individual VPNs for multiple accounts is too much work, and VPNs bring more latency.

- ☒ Configure AWS Transit Gateway between the accounts. Assign DX to the transit gateway and route network traffic to the on-premises servers.

Transit Gateway simplifies the network by acting as a hub for all VPCs and the on-premises network. Instead of multiple DX or VPN connections, you're using one centralized DX connection with Transit

Gateway. You can easily add new accounts or VPCs without reconfiguring DX or VPN connections.

Overall Explanation:

AWS Transit Gateway offers the least operational overhead and is the most scalable and cost-effective solution. It centralizes network management, eliminating the need for separate DX connections or VPNs for each new account.

[View Discussions](#)[Ask AI](#)

Question 28

Not Attempted

Cost Optimization



A company hosts its main public web application in one AWS Region across multiple Availability Zones. The application uses an Amazon EC2 Auto Scaling group and an Application Load Balancer (ALB). A web development team needs a cost-optimized compute solution to improve the company's ability to serve dynamic content globally to millions of customers. Which solution will meet these requirements?

- ☒ Create an Amazon CloudFront distribution. Configure the existing ALB as the origin.
CloudFront is a CDN that caches content at edge locations, reducing latency for users globally. Using the ALB as the origin allows CloudFront to distribute dynamic content from the web application. This is a cost-effective way to serve dynamic content globally.
- ☐ Use Amazon Route 53 to serve traffic to the ALB and EC2 instances based on the geographic location of each customer.
While Route 53 Geo DNS can route traffic based on location, it doesn't cache content like a CDN, and therefore doesn't reduce latency as effectively as CloudFront for dynamic content.
- ☐ Create an Amazon S3 bucket with public read access enabled. Migrate the web application to the S3 bucket. Configure the S3 bucket for website hosting.
S3 website hosting is suitable for static content but not ideal for dynamic content as it requires significant re-architecting of the application and doesn't inherently provide global content delivery optimization.

- ⊗ Use AWS Direct Connect to directly serve content from the web application to the location of each customer.

Direct Connect establishes a dedicated network connection to AWS. It is typically used for hybrid environments and transferring large datasets. It does not provide global content delivery optimization and is not suitable for serving dynamic content to millions of users.

Overall Explanation:

Creating an Amazon CloudFront distribution with the existing ALB as the origin is the most cost-effective and efficient way to serve dynamic content globally. CloudFront's caching capabilities at edge locations minimize latency and improve the user experience. Route 53 Geo DNS offers location-based routing but does not offer the caching benefits of a CDN. S3 website hosting is not suitable for dynamic content. Direct Connect provides a direct network connection but does not offer global content delivery optimization.

 View Discussions

 Ask AI

Question 29

Not Attempted

Storage



A company stores user data in AWS. The data is accessed continuously, with peak usage during business hours. Access patterns vary significantly, with some data infrequently accessed for months. A solutions architect needs a cost-effective storage solution that provides high durability and availability. Which storage solution best meets these requirements?

- ⊗ Amazon S3 Standard

Amazon S3 Standard offers high durability and availability but may not be the most cost-effective option for data with varying access patterns, particularly infrequently accessed data.

- ✓ Amazon S3 Intelligent-Tiering

Amazon S3 Intelligent-Tiering automatically moves data between frequent, infrequent, and archive access tiers based on changing access patterns. This optimizes costs while maintaining high durability and availability.

- ⊗ Amazon S3 Glacier Deep Archive

Amazon S3 Glacier Deep Archive is designed for long-term archiving and has the lowest storage cost, but retrieval times can be several hours, making it unsuitable for continuously accessed data.

- ⊗ Amazon S3 One Zone-Infrequent Access (S3 One Zone-IA)

Amazon S3 One Zone-IA is designed for infrequently accessed data with lower availability as it only stores data in a single Availability Zone. This doesn't meet the high availability requirement.

Overall Explanation:

Amazon S3 Intelligent-Tiering is the best solution because it automatically optimizes storage costs by moving data between different access tiers based on access patterns, ensuring cost-effectiveness while maintaining high durability and availability. S3 Standard is more expensive for infrequently accessed data. Glacier Deep Archive has long retrieval times, and S3 One Zone-IA doesn't offer the required high availability.

[View Discussions](#)[Ask AI](#)

Question 30

Not Attempted

Storage



A company is testing an application that runs on an Amazon EC2 Linux instance. A single 500 GB Amazon Elastic Block Store (Amazon EBS) General Purpose SSD (gp2) volume is attached to the EC2 instance. The company will deploy the application on multiple EC2 instances in an Auto Scaling group. All instances require access to the data that is stored in the EBS volume. The company needs a highly available and resilient solution that does not introduce significant changes to the application's code. Which solution will meet these requirements?

- ☒ Provision an EC2 instance that uses NFS server software. Attach a single 500 GB gp2 EBS volume to the instance.
This solution introduces a single point of failure and does not automatically scale to meet demand. Managing an NFS server adds operational overhead.
- ☒ Provision an Amazon FSx for Windows File Server file system. Configure the file system as an SMB file store within a single Availability Zone.
Amazon FSx for Windows File Server is designed for Windows-based applications, and the scenario uses EC2 Linux instances.
- ☒ Provision an EC2 instance with two 250 GB Provisioned IOPS SSD EBS volumes.
This doesn't address the core requirement of making the data accessible to multiple EC2 instances simultaneously in a scalable and resilient manner. It also unnecessarily increases complexity and cost.

- ✓ Provision an Amazon Elastic File System (Amazon EFS) file system. Configure the file system to use General Purpose performance mode.

Amazon EFS is a network file system designed for high availability and scalability and can automatically scale to accommodate the needs of multiple EC2 instances in an Auto Scaling group. It integrates seamlessly with Linux-based EC2 instances and requires minimal code changes. General Purpose performance mode is suitable for most application workloads.

Overall Explanation:

The correct answer is to use Amazon EFS. EFS provides a shared file system that multiple EC2 instances can access simultaneously. It's highly available, scales automatically, and integrates well with Linux-based EC2 instances with minimal code changes. The General Purpose performance mode offers a balance between cost and performance suitable for typical application workloads. Options A, B, and C are incorrect because they are either not highly available and scalable, not designed for Linux, or do not provide a shared file system solution.

 View Discussions

 Ask AI

Question 31

Not Attempted

Networking



A company recently launched a new application for its customers. The application runs on multiple Amazon EC2 instances across two Availability Zones. End users use TCP to communicate with the application. The application must be highly available and must automatically scale as the number of users increases. Which combination of steps will meet these requirements MOST cost-effectively?

- ✓ Add a Network Load Balancer in front of the EC2 instances.

A Network Load Balancer (NLB) is designed for TCP traffic and provides high performance and scalability. It operates at layer 4 and is more efficient for TCP than an ALB, thus making it more cost-effective.

- ✓ Configure an Auto Scaling group for the EC2 instances.

An Auto Scaling group automatically adjusts the number of EC2 instances based on demand, ensuring high availability and scalability. This allows the application to scale up or down as needed, optimizing costs.

- ✗ Add an Application Load Balancer in front of the EC2 instances.

While an Application Load Balancer (ALB) can handle TCP traffic, it's designed for HTTP/HTTPS traffic and operates at layer 7. For simple TCP traffic, it is less cost-effective than an NLB.

⊗ **Manually add more EC2 instances for the application.**

Manually adding EC2 instances is not cost-effective or scalable. It requires constant monitoring and intervention, and it doesn't provide the automated scaling benefits of an Auto Scaling group.

⊗ **Add a Gateway Load Balancer in front of the EC2 instances.**

A Gateway Load Balancer (GWLB) is primarily used for third-party virtual appliances. It is not suited for load balancing general application traffic to EC2 instances in this scenario and would be unnecessarily complex and costly.

Overall Explanation:

Using a Network Load Balancer (NLB) is the most cost-effective solution for TCP traffic as it operates at layer 4, reducing overhead compared to an Application Load Balancer (ALB).

Combining the NLB with an Auto Scaling group ensures the application scales automatically based on demand, optimizing resource utilization and costs while maintaining high availability. Manually adding EC2 instances is not a scalable or cost-effective approach. A Gateway Load Balancer is also unsuitable for this general application load balancing scenario.

 [View Discussions](#)

 [Ask AI](#)

Question 32

Not Attempted

Security



A company is designing the architecture for a new mobile app that uses the AWS Cloud. The company uses organizational units (OUs) in AWS Organizations to manage its accounts. The company wants to tag Amazon EC2 instances with data sensitivity by using values of sensitive and nonsensitive. IAM identities must not be able to delete a tag or create instances without a tag. Which combination of steps will meet these requirements? (Choose two.)

⊗ **In Organizations, create a new tag policy that specifies the data sensitivity tag key and the required values. Enforce the tag values for the EC2 instances. Attach the tag policy to the appropriate OU.**

Tag policies cannot enforce tag values directly. They can only specify allowed tag keys and values.

- ⊗ In Organizations, create a new service control policy (SCP) that specifies the data sensitivity tag key and the required tag values. Enforce the tag values for the EC2 instances. Attach the SCP to the appropriate OU.
SCPs cannot directly enforce tag values; they can only allow or deny actions based on tags.
- ⊗ Create a tag policy to deny running instances when a tag key is not specified. Create another tag policy that prevents identities from deleting tags. Attach the tag policies to the appropriate OU.
Tag policies in AWS Organizations can't deny actions, such as launching untagged instances or deleting tags.
- ✔ Create a service control policy (SCP) to deny creating instances when a tag key is not specified. Create another SCP that prevents identities from deleting tags. Attach the SCPs to the appropriate OU.
This is a valid approach. SCPs can deny actions based on the presence or absence of tags and can prevent tag deletion.
- ⊗ Create an AWS Config rule to check if EC2 instances use the data sensitivity tag and the specified values. Configure an AWS Lambda function to delete the resource if a noncompliant resource is found.
This is an option, but it doesn't prevent resources from being created without the tags initially. It only cleans up afterwards, which may not meet the requirement.

Overall Explanation:

The correct answers are to create SCPs to prevent launching EC2 instances without the required tags and to prevent deleting the tags. SCPs are used within AWS Organizations to manage permissions across accounts and can deny actions based on tag presence. Tag policies cannot enforce tag values directly nor can they deny actions. AWS Config can be used to detect non-compliant resources, but this does not prevent the creation of resources that do not have the correct tags.

[View Discussions](#)[Ask AI](#)

Question 33

Not Attempted

[Databases](#)

A company runs database workloads on AWS that are the backend for the company's customer portals. The company runs a Multi-AZ database cluster on Amazon RDS for PostgreSQL. The

company needs to implement a 30-day backup retention policy. The company currently has both automated RDS backups and manual RDS backups. The company wants to maintain both types of existing RDS backups that are less than 30 days old. Which solution will meet these requirements MOST cost-effectively?

- ☐ ⓧ Configure the RDS backup retention policy to 30 days for automated backups by using AWS Backup. Manually delete manual backups that are older than 30 days.
AWS Backup introduces additional costs, making this option less cost-effective.
- ☐ ⓧ Disable RDS automated backups. Delete automated backups and manual backups that are older than 30 days. Configure the RDS backup retention policy to 30 days for automated backups.
Disabling RDS automated backups is not necessary and could impact recovery time in case of failures. Also deleting automated backups before setting retention is not optimal.
- ☒ ⓧ Configure the RDS backup retention policy to 30 days for automated backups. Manually delete manual backups that are older than 30 days.
This solution leverages the built-in RDS backup retention policy for automated backups and relies on manual deletion for manual backups, which is the most cost-effective approach.
- ☐ ⓧ Disable RDS automated backups. Delete automated backups and manual backups that are older than 30 days automatically by using AWS CloudFormation. Configure the RDS backup retention policy to 30 days for automated backups.
Using CloudFormation to delete backups adds unnecessary complexity and cost, and disabling automated backups is not recommended.

Overall Explanation:

The most cost-effective solution is to configure the RDS backup retention policy to 30 days for automated backups, which utilizes the native functionality without incurring additional costs. Manual deletion of manual backups older than 30 days, although requiring some operational overhead, is the least expensive way to manage them. Options A and D involve additional services (AWS Backup and CloudFormation) that add to the cost. Option B is suboptimal as it disables automated backups which is not ideal for RTO/RPO, it also involves deleting automated backups before setting the retention policy.

[View Discussions](#)[Ask AI](#)

Question 34

Not Attempted

Storage



A company is implementing a shared storage solution for a gaming application that is hosted in an on-premises data center. The company needs the ability to use Lustre clients to access data. The solution must be fully managed. Which solution meets these requirements?

- ☐ Create an AWS Storage Gateway file gateway. Create a file share that uses the required client protocol. Connect the application server to the file share.

AWS Storage Gateway does not natively support Lustre clients.

- ☐ Create an Amazon EC2 Windows instance. Install and configure a Windows file share role on the instance. Connect the application server to the file share.

A Windows file share does not support Lustre clients.

- ☐ Create an Amazon Elastic File System (Amazon EFS) file system, and configure it to support Lustre. Attach the file system to the origin server. Connect the application server to the file system.

Amazon EFS does not natively support Lustre clients.

- ☒ Create an Amazon FSx for Lustre file system. Attach the file system to the origin server. Connect the application server to the file system.

Amazon FSx for Lustre provides a fully managed file system that is optimized for Lustre workloads, which allows Lustre clients to seamlessly connect to the file system. However, because the application is on-premise, some extra configuration is necessary for connectivity and low latency. FSx for Lustre can connect to an on-premises datacenter via VPN or Direct Connect.

Overall Explanation:

Amazon FSx for Lustre provides a fully managed file system that is optimized for high-performance computing workloads, including Lustre clients. It enables the use of Lustre clients to access data in a managed and scalable manner. While the application is on-premise, connectivity to FSx for Lustre can be established using VPN or Direct Connect.

[View Discussions](#)[Ask AI](#)

Question 35

Not Attempted

Storage



A company is migrating a legacy application to AWS. This application relies on NFS for communication with an on-premises storage solution to store application data. Due to application limitations, NFS is the only supported communication protocol. Which AWS storage service should a solutions architect recommend to ensure continued functionality after the migration?

☐ AWS DataSync

AWS DataSync is a data transfer service and does not provide an NFS file system.

☐ Amazon Elastic Block Store (Amazon EBS)

Amazon EBS provides block storage and does not natively support NFS. While you could create an NFS server on an EC2 instance with EBS, it's not a fully managed solution.

☒ Amazon Elastic File System (Amazon EFS)

Amazon EFS provides a fully managed NFS file system, making it the suitable choice for applications requiring NFS access.

☐ Amazon EMR File System (Amazon EMRFS)

Amazon EMRFS is a file system used with Amazon EMR for big data processing and is not designed to be a general-purpose NFS server.

Overall Explanation:

Amazon EFS is the correct choice because it is a fully managed NFS file system. This allows the legacy application to continue using NFS for file storage after migration to AWS without requiring any application code changes. The other options do not provide native NFS support or are intended for different use cases.

 [View Discussions](#)

 [Ask AI](#)

Question 36

Not Attempted

Serverless



A company uses GPS trackers to document the migration patterns of thousands of sea turtles. The trackers check every 5 minutes to see if a turtle has moved more than 100 yards (91.4 meters). If a turtle has moved, its tracker sends the new coordinates to a web application running on three Amazon EC2 instances that are in multiple Availability Zones in one AWS Region. Recently, the web application was overwhelmed while processing an unexpected

volume of tracker data. Data was lost with no way to replay the events. A solutions architect must prevent this problem from happening again and needs a solution with the least operational overhead. What should the solutions architect do to meet these requirements?

- ☐ Create an Amazon S3 bucket to store the data. Configure the application to scan for new data in the bucket for processing.

Storing data in S3 and having the application scan it is inefficient and adds operational overhead. It does not provide a queuing mechanism to handle bursts of data.

- ☐ Create an Amazon API Gateway endpoint to handle transmitted location coordinates. Use an AWS Lambda function to process each item concurrently.

While API Gateway and Lambda can handle high volumes, they may not provide a reliable buffer against data loss in case of application overload and may lead to throttling. This solution also has higher operational overhead compared to SQS.

- ☒ Create an Amazon Simple Queue Service (Amazon SQS) queue to store the incoming data. Configure the application to poll for new messages for processing.

Amazon SQS provides a reliable and scalable way to decouple the components of the application. By using a queue, the incoming tracker data can be buffered, ensuring that if the web application is overwhelmed with data, the messages will still be retained in the queue until they can be processed. SQS is a fully managed service, thus it reduces operational overhead.

- ☐ Create an Amazon DynamoDB table to store transmitted location coordinates. Configure the application to query the table for new data for processing. Use TTL to remove data that has been processed.

Using DynamoDB as a queue is not its intended purpose and introduces unnecessary complexity. Polling a DynamoDB table for new data is inefficient and can lead to high read costs. TTL will remove data, but it doesn't guarantee processing.

Overall Explanation:

Amazon SQS is the best solution because it acts as a buffer between the GPS trackers and the web application. This ensures that even if there's a sudden spike in data, no data is lost. The web application can then process the data at its own pace by polling the queue. Also, SQS is a fully managed service, minimizing the operational overhead.

 [View Discussions](#)

 [Ask AI](#)

A company's software development team requires an Amazon RDS Multi-AZ cluster to serve as a backend for a desktop client deployed on-premises. The desktop client needs direct connectivity to the RDS cluster. The company mandates that the development team can only connect to the cluster using the client when they are physically present in the office. Which solution provides the required connectivity with the HIGHEST level of security?

- ☐ Create a VPC and two public subnets. Create the RDS cluster in the public subnets. Use AWS Site-to-Site VPN with a customer gateway in the company's office.

Placing the RDS cluster in public subnets exposes it to the internet, even with a VPN. This increases the attack surface and is less secure.

- ☒ Create a VPC and two private subnets. Create the RDS cluster in the private subnets. Use AWS Site-to-Site VPN with a customer gateway in the company's office.

This is the most secure option. Private subnets isolate the RDS cluster from the internet, and the VPN provides a secure, encrypted connection from the office.

- ☐ Create a VPC and two private subnets. Create the RDS cluster in the private subnets. Use RDS security groups to allow the company's office IP ranges to access the cluster.

While using private subnets is good, relying solely on security groups based on IP ranges is less secure than a VPN. IP addresses can be spoofed or changed, making this approach less robust.

- ☐ Create a VPC and two public subnets. Create the RDS cluster in the public subnets. Create a cluster user for each developer. Use RDS security groups to allow the users to access the cluster.

This is the least secure option. Exposing the RDS cluster in public subnets and relying on individual user accounts and security groups provides a much larger attack surface.

Overall Explanation:

The most secure solution involves creating a VPC with private subnets for the RDS cluster and establishing a secure AWS Site-to-Site VPN connection from the company's office using a customer gateway. This approach ensures that the RDS cluster is not directly exposed to the internet and that access is only permitted through an encrypted tunnel when the development team is connected from the office.

 View Discussions

 Ask AI

A solutions architect is designing an application for batch processing of large datasets. Input data resides in Amazon S3, and output data will be stored in a different S3 bucket. The application processes data by transferring it over the network between multiple Amazon EC2 instances. To minimize overall data transfer costs, what strategy should the architect implement?

☐ Place all the EC2 instances in an Auto Scaling group.

Placing EC2 instances in an Auto Scaling group primarily addresses scalability and availability, not data transfer costs.

☐ Place all the EC2 instances in the same AWS Region.

While placing EC2 instances and S3 buckets in the same region helps, it doesn't eliminate data transfer costs entirely. Data transfer between different Availability Zones within the same region can still incur costs.

☒ Place all the EC2 instances in the same Availability Zone.

Placing all EC2 instances in the same Availability Zone minimizes data transfer costs, as data transfer between instances within the same AZ is free. Additionally, data transfer between EC2 instances and S3 within the same AZ is free.

☐ Place all the EC2 instances in private subnets in multiple Availability Zones.

Placing EC2 instances in multiple Availability Zones incurs data transfer costs between the zones, which contradicts the requirement to reduce data transfer costs.

Overall Explanation:

To reduce overall data transfer costs, the solutions architect should place all the EC2 instances in the same Availability Zone. Data transfer between EC2 instances in the same Availability Zone is free. Moreover, there is no charge for data transfer between EC2 and S3 within the same AZ.

 View Discussions

 Ask AI

Question 39

Not Attempted

Security



A company hosts a multi-tier web application that uses an Amazon Aurora MySQL DB cluster for storage. The application tier is hosted on Amazon EC2 instances. The company's IT security guidelines mandate that the database credentials be encrypted and rotated every 14 days.

What should a solutions architect do to meet this requirement with the LEAST operational effort?

- ✔ Create a new AWS Key Management Service (AWS KMS) encryption key. Use AWS Secrets Manager to create a new secret that uses the KMS key with the appropriate credentials. Associate the secret with the Aurora DB cluster. Configure a custom rotation period of 14 days.
- ✘ Create two parameters in AWS Systems Manager Parameter Store: one for the user name as a string parameter and one that uses the SecureString type for the password. Select AWS Key Management Service (AWS KMS) encryption for the password parameter, and load these parameters in the application tier. Implement an AWS Lambda function that rotates the password every 14 days.

AWS Secrets Manager allows for automated rotation of credentials using KMS for encryption and supports custom rotation periods like 14 days. It integrates directly with Aurora for easy credential management.

This approach requires implementing a custom rotation mechanism with Lambda, increasing operational overhead.

- ✘ Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon Elastic File System (Amazon EFS) file system. Mount the EFS file system in all EC2 instances of the application tier. Restrict the access to the file on the file system so that the application can read the file and that only super users can modify the file. Implement an AWS Lambda function that rotates the key in Aurora every 14 days and writes new credentials into the file.

This method involves more complex setup and management, including EFS mounting and access control, along with custom rotation logic.

- ✘ Store a file that contains the credentials in an AWS Key Management Service (AWS KMS) encrypted Amazon S3 bucket that the application uses to load the credentials. Download the file to the application regularly to ensure that the correct credentials are used. Implement an AWS Lambda function that rotates the Aurora credentials every 14 days and uploads these credentials to the file in the S3 bucket.

This option introduces S3 object storage and download overhead, as well as the need for custom rotation logic and credential management in the application.

Overall Explanation:

AWS Secrets Manager provides the most straightforward solution with the least operational effort. It offers built-in integration with Aurora for credential management and

allows for custom rotation periods, utilizing KMS for encryption. Options B, C, and D all require manual implementation of rotation and management of credentials, adding unnecessary complexity.

[View Discussions](#)[Ask AI](#)

Question 40

Not Attempted

Serverless



A streaming media company is rebuilding its infrastructure to accommodate increasing demand for video content consumed daily. The company needs to process terabyte-sized videos to block inappropriate content. Video processing can take up to 20 minutes. The company requires a solution that will scale with demand and remain cost-effective. Which solution BEST meets these requirements?

- ☐ Use AWS Lambda functions to process videos. Store video metadata in Amazon DynamoDB. Store video content in Amazon S3 Intelligent-Tiering.

AWS Lambda functions have execution time limits, making them unsuitable for processing videos that take up to 20 minutes. Also, Lambda might not be appropriate for TB-sized files.

- ☒ Use Amazon Elastic Container Service (Amazon ECS) and AWS Fargate to implement microservices to process videos. Store video metadata in Amazon Aurora. Store video content in Amazon S3 Intelligent-Tiering.

This solution uses ECS with Fargate for scalable, serverless container execution, suitable for processing videos. Aurora provides a highly available database for metadata. S3 Intelligent-Tiering optimizes storage costs.

- ☐ Use Amazon EC2 instances in an Auto Scaling group behind an Application Load Balancer (ALB) to process videos. Store video content in Amazon S3 Standard. Use Amazon Simple Queue Service (Amazon SQS) for queuing and to decouple processing tasks.

While EC2 Auto Scaling can handle demand, it involves more management overhead compared to Fargate. S3 Standard is less cost-effective than S3 Intelligent-Tiering for large video files. Using EC2 increases the operational complexity and cost.

- ☐ Deploy a containerized video processing application on Amazon Elastic Kubernetes Service (Amazon EKS) on Amazon EC2. Store video metadata in Amazon RDS in a single Availability Zone. Store video content in Amazon S3 Glacier Deep Archive.

EKS on EC2 introduces management overhead. RDS in a single Availability Zone lacks high availability. S3 Glacier Deep Archive is designed for long-term archival and is not suitable for frequently accessed video content.

Overall Explanation:

The most cost-effective and scalable solution is to use Amazon ECS with Fargate to implement microservices for video processing. Fargate allows running containers without managing EC2 instances, scaling automatically based on demand. Amazon Aurora is used to store metadata, providing high availability, and Amazon S3 Intelligent-Tiering is used for storing video content, optimizing storage costs. Lambda is not suitable for long-running tasks. EC2 auto scaling has more management overhead.

[View Discussions](#)[Ask AI](#)

Question 41

Not Attempted

[Compute](#)

A company is migrating its on-premises application, which is running on a Kubernetes cluster, to the AWS Cloud due to a significant increase in customers that their existing infrastructure cannot support. They plan to use Amazon Elastic Kubernetes Service (Amazon EKS) but want to minimize the operational overhead associated with managing the underlying compute infrastructure. Which solution provides the LEAST operational overhead?

- ☐ Use a self-managed node group to supply compute capacity. Deploy the application to the new EKS cluster.

Self-managed node groups require you to manage the EC2 instances yourself, increasing operational overhead.

- ☐ Use managed node groups to supply compute capacity. Deploy the application to the new EKS cluster.

Managed node groups simplify node management but still require some level of EC2 instance management.

- ☒ Use AWS Fargate to supply compute capacity. Create a Fargate profile. Use the Fargate profile to deploy the application.

AWS Fargate is a serverless compute engine for containers. It removes the need to provision and manage EC2 instances, thus minimizing operational overhead. By defining a Fargate profile, you can specify which pods run on Fargate.

- ⊗ Use managed node groups with Karpenter to supply compute capacity. Deploy the application to the new EKS cluster.

While Karpenter automates node provisioning, using it with managed node groups still involves managing the underlying EC2 infrastructure to some extent, resulting in more operational overhead compared to Fargate.

Overall Explanation:

AWS Fargate offers the least operational overhead because it is a serverless compute option for containers. It eliminates the need to manage the underlying EC2 instances. Creating a Fargate profile allows deploying applications without worrying about node management. Therefore, this is the correct answer. The other options, A, B, and D, all require some level of compute infrastructure management.

 View Discussions

 Ask AI

Question 42

Not Attempted

Databases



A company is developing a new social media application that needs a relational database to store user profiles, posts, and relationships. The database must automatically scale based on application traffic, provide backup and recovery capabilities, and be cost-effective. Which of the following AWS database solutions BEST meets these requirements?

- ⊗ Deploy a self-managed MySQL database on Amazon EC2 instances using Spot Instances for cost optimization. Configure automated backups to Amazon S3.

Using Spot Instances for a database is risky because they can be terminated with little notice, which can lead to data loss or corruption. Also, self-managing a database adds operational overhead.

- ⊗ Use Amazon RDS for MySQL with on-demand capacity mode and General Purpose SSD storage. Configure automatic backups with a retention period of 7 days.

While RDS simplifies database management, on-demand capacity might not be the most cost-effective option for fluctuating workloads. Aurora Serverless offers better cost optimization for unpredictable traffic.

- ✔ Use Amazon Aurora Serverless for the database. Utilize serverless capacity scaling and configure automated backups to Amazon S3.

Aurora Serverless is a fully managed relational database that automatically scales with traffic and includes integrated automated backups, making it a cost-effective choice for variable workloads.

- ⊗ Deploy a self-managed NoSQL database (like Cassandra) on Amazon EC2 instances using Reserved Instances for cost optimization. Configure automated backups directly to Amazon S3 Glacier Flexible Retrieval.

The application requires a relational database, and a NoSQL database is not suitable. Also, self-managing a database adds operational overhead.

Overall Explanation:

Amazon Aurora Serverless is the most suitable solution because it is a fully managed relational database service that automatically scales capacity based on application traffic. This eliminates the need to provision and manage database instances manually. The service also offers automated backups and restores, reducing the operational overhead and ensuring data durability. Spot Instances are not suitable for stateful applications like databases. Using RDS with on-demand capacity is more expensive than using Aurora Serverless, which scales up and down according to the needs of the application. NoSQL databases are not suitable as the application requires a structured database.

 View Discussions

 Ask AI

Question 43

Not Attempted

Networking



A company's legacy web application runs on an Amazon EC2 instance in a public subnet of a VPC. The application stores customer images on a locally attached Amazon EBS volume. Images are backed up nightly to an Amazon S3 bucket. Currently, uploads to S3 use the public endpoint. A solutions architect needs to ensure S3 traffic does not use the public endpoint. Which solution meets these requirements?

- ✓ Create a gateway VPC endpoint for the S3 bucket with necessary permissions for the VPC. Configure the subnet route table to use the gateway VPC endpoint.

A gateway VPC endpoint provides private connectivity between resources in a VPC and S3, ensuring traffic stays within the AWS network and doesn't use public IPs.

- ⊗ Move the S3 bucket inside the VPC. Configure the subnet route table to access the S3 bucket through private IP addresses.

Amazon S3 is a regional service and cannot be moved inside a VPC.

- ⊗ Create an Amazon S3 access point for the Amazon EC2 instance inside the VPC. Configure the web application to upload by using the Amazon S3 access point.
S3 access points simplify access control but do not, by themselves, ensure private connectivity to S3.
- ⊗ Configure an AWS Direct Connect connection between the VPC that has the Amazon EC2 instance and Amazon S3 to provide a dedicated network path.
While Direct Connect provides dedicated connectivity, it is more complex and expensive than using a gateway VPC endpoint for private S3 access.

Overall Explanation:

Creating a gateway VPC endpoint for S3 ensures that traffic between the EC2 instance in the VPC and the S3 bucket stays within the AWS network, avoiding the public internet. The route table configuration ensures the EC2 instance uses the endpoint. The other options are either incorrect (S3 cannot reside within a VPC) or offer solutions that don't directly address the requirement of private S3 access as effectively and efficiently as a gateway endpoint.

[View Discussions](#)[Ask AI](#)

Question 44

Not Attempted

[Databases](#)

A company is developing an e-commerce website prototype on AWS. The architecture includes an Application Load Balancer, an Auto Scaling group of Amazon EC2 instances for web servers, and an Amazon RDS for MySQL DB instance running in a Single-AZ configuration. During product catalog searches, the website's response time is slow, and the database CPU utilization spikes. The product catalog, stored in several MySQL tables, is infrequently updated. A solutions architect needs to improve website performance during these searches. What should the solutions architect recommend?

- ⊗ Migrate the product catalog to an Amazon Redshift database. Use the COPY command to load the product catalog tables.
Redshift is optimized for analytical queries on large datasets. It's not suited for transactional or low-latency queries typical of product catalog searches.
- ✔ Implement an Amazon ElastiCache for Redis cluster to cache the product catalog. Use lazy loading to populate the cache.

ElastiCache for Redis, being an in-memory caching service, significantly reduces database load by caching frequently accessed data. Lazy loading ensures that data is only added to the cache when it's first requested, optimizing resource utilization.

- ⊗ Add an additional scaling policy to the Auto Scaling group to launch additional EC2 instances when database response is slow.

Adding more web servers does not alleviate the bottleneck at the database level. The problem lies in the database's inability to handle the search queries efficiently.

- ⊗ Turn on the Multi-AZ configuration for the DB instance. Configure the EC2 instances to throttle the product catalog queries that are sent to the database.

Enabling Multi-AZ improves availability but doesn't address the CPU utilization issue directly. Throttling queries degrades user experience and should be avoided if possible. Furthermore, Multi-AZ only provides high availability not performance boost.

Overall Explanation:

The best solution is to implement an Amazon ElastiCache for Redis cluster and cache the product catalog with a lazy loading strategy. This reduces the load on the MySQL database, resulting in faster response times during product catalog searches. Redshift is not optimized for these types of reads. Additional EC2 instances won't help, as the DB is the bottleneck. Multi-AZ doesn't improve performance and throttling negatively impacts user experience.

[View Discussions](#)[Ask AI](#)[← Back to Exam](#)[Marketing](#)[Company](#)[Affiliate Program](#)[Udemy Alternative](#)

[Unlimited Access](#)

[Whizlabs Alternative](#)

[Business Unlimited Access](#)

[TutorialsDojo Alternative](#)

[Support](#)

[Legal](#)

[Contact Us](#)

[Terms of service](#)

team@itcertifications.io

[Privacy policy](#)

[Refund policy](#)

[Shipping policy](#)



© 2026, ITcertifications Technologies. All rights reserved.